

1. 1 管理线程
2. 1.1 线程管理基础
3. 1.2 向线程函数传递参数
 1. 1.3 转移线程所有权
4. 2 线程间共享数据
 1. 2.1 使用互斥量
 2. 2.2 其他类型互斥量
 1. 2.2.2 recursive_mutex
 2. 2.2.3 shared_mutex
 3. 2.3 各种锁介绍
 1. 2.3.1 std::unique_lock -- 更加灵活的锁
 2. 2.3.2 单次调用加锁
 3. 2.3.3 同时加锁多个互斥量
 1. 2.3.3.1 std::lock
 2. 2.3.3.2 std::scoped_lock(C++ 17)
 4. 2.3.4 共享锁(C++ 14)
5. 3 同步并发操作
 1. 3.1 条件变量使用方式
6. 3.2 使用期待处理一次性事件
 1. 3.2.1 std::future
 2. 3.2.2 std::shared_future
 3. 3.3 更多的使用 future的方式
 1. 3.3.1 std::package_task
 2. 3.3.2 std::promise
7. 4 原子操作
 1. 4.1 内存顺序
 2. 4.2 概念解释
 3. 4.3 memory order
8. 5 参考

- 1 管理线程
- 1.1 线程管理基础
- 1.2 向线程函数传递参数
 - 1.3 转移线程所有权
- 2 线程间共享数据
 - 2.1 使用互斥量
 - 2.2 其他类型互斥量

- 2.2.2 recursive_mutex
- 2.2.3 shared_mutex
- 2.3 各种锁介绍
 - 2.3.1 std::unique_lock -- 更加灵活的锁
 - 2.3.2 单次调用加锁
 - 2.3.3 同时加锁多个互斥量
 - 2.3.3.1 std::lock
 - 2.3.3.2 std::scoped_lock(C++ 17)
 - 2.3.4 共享锁(C++ 14)
- 3 同步并发操作
 - 3.1 条件变量使用方式
- 3.2 使用期待处理一次性事件
 - 3.2.1 std::future
 - 3.2.2 std::shared_future
 - 3.3 更多的使用 future 的方式
 - 3.3.1 std::package_task
 - 3.3.2 std::promise
- 4 原子操作
 - 4.1 内存顺序
 - 4.2 概念解释
 - 4.3 memory order
- 5 参考

1 管理线程

1.1 线程管理基础

C++线程库启动线程，可以归结为构造std::thread对象,如下是一个最简单的例子：

```
void do_some_work();

std::thread my_thread(do_some_work);
```

std::thread可以接受callable类型构造。

```
class baz
{
public:
```

```

void operator>()()
{
    for (int i = 0; i < 5; ++i) {
        std::cout << "Thread 4 executing\n";
        ++n;
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}
int n = 0;
};

int main() {
    baz b;
    std::thread t1(b); // t6 在对象 b 的副本上运行 baz::operator()
    t1.join(); //等待线程完成
}

```

如果想要分离一个线程，可以在线程启动后，直接使用`detach()`进行分离。如果打算等待对应线程，则需要细心挑选调用`join()`的位置。

使用`detach()`会让线程在后台运行，这就意味着主线程不能与之产生直接交互。也就是说，不会等待这个线程结束；如果线程分离，那么就不可能有`std::thread`对象能引用它，分离线程的确在后台运行，所以分离线程不能被加入。

1.2 向线程函数传递参数

如下是几种向线程传递参数的方式：

```

void f1(int n)
{
    for (int i = 0; i < 5; ++i) {
        std::cout << "Thread 1 executing\n";
        ++n;
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}

void f2(int& n)
{
    for (int i = 0; i < 5; ++i) {
        std::cout << "Thread 2 executing\n";
        ++n;
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}

class foo
{
public:
    void bar()

```

```

    {
        for (int i = 0; i < 5; ++i) {
            std::cout << "Thread 3 executing\n";
            ++n;
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
    }
    int n = 0;
};

int main()
{
    int n = 0;
    foo f;
    baz b;
    std::thread t1; // t1 不是线程
    std::thread t2(f1, n + 1); // 按值传递
    std::thread t3(f2, std::ref(n)); // 按引用传递
    std::thread t4(&foo::bar, &f); // t5 在对象 f 上运行 foo::bar()
    t1.join();
    t2.join();
    t3.join();
    t4.join();
}

```

1.3 转移线程所有权

先看一个实例

```

int main()
{
    int n = 0;
    std::thread t1(f1, n + 1); // 按值传递
    std::thread t2(std::move(t3)); // t4 现在运行 f2() 。 t3 不再是线程
    t2.join();
}

```

1. 创建一个线程，并在函数中转移所有权(如上面的例子)
2. 要写一个在后台启动线程的函数，想通过新线程返回的所有权去调用这个函数,如下例：

```

std::thread f()
{
    void some_function();
    return std::thread(some_function);
}

std::thread g()

```

```
{  
    void some_other_function(int);  
    std::thread t(some_other_function,42);  
    return t;  
}
```

2 线程间共享数据

通常我们使用锁保护线程间共享数据，这也是最基本的方式。

当访问共享数据前，使用互斥量将相关数据锁住，再当访问结束后，再将数据解锁。线程库需要保证，当一个线程使用特定互斥量锁住共享数据时，其他的线程想要访问锁住的数据，都必须等到之前那个线程对数据进行解锁后，才能进行访问。这就保证了所有线程能看到共享数据，而不破坏不变量。

2.1 使用互斥量

C++提供 `std::mutex` 创建互斥量，通过调用 `lock()` 上锁，`unlock()` 解锁。

为方便使用，C++提供RAII语法的模板类 `std::lock_guard()`，可以在离开锁作用域是自动解锁。其有以下特点：

- 创建即加锁，作用域结束自动析构并解锁，无需手动解锁
- 不能中途解锁
- 不能复制

```
int g_i = 0;  
std::mutex g_i_mutex;  
  
void safe_increment() {  
    std::lock_guard lock(g_i_mutex); //safe_increment结束时自动解锁  
    g_i++;  
}
```

2.2 其他类型互斥量

接下来介绍另外两种互斥量：`std::recursive_mutex`和 `shared_mutex`

2.2.2 recursive_mutex

`recursive_mutex`和 `mutex`行为几乎一致，区别在于提供排他性递归所有权语义，已经获得一个递归互斥体的所有权的线程允许在同一个互斥体上再次调用 `lock()`和 `try_lock()`，调用线程调用 `unlock`的次数应该等于获的这个递归互斥锁的次数，在匹配次数时解锁。

比如函数A需要获取锁`mutex`，函数B也需要获取锁`mutex`，同时函数A中还会调用函数B。如果使用 `std::mutex`必然会造成死锁。但是使用 `std::recursive_mutex`就可以解决这个问题

2.2.3 shared_mutex

`shared_mutex` 类是一个同步原语，可用于保护共享数据不被多个线程同时访问。与便于独占访问的其他互斥类型不同，`shared_mutex` 拥有二个访问级别：

- 共享 - 多个线程能共享同一互斥的所有权。
- 独占性 - 仅一个线程能占有互斥。

若一个线程已获取独占性锁（通过 `lock` 、 `try_lock` ），则无其他线程能获取该锁（包括共享的）。仅当任何线程均未获取独占性锁时，共享锁能被多个线程获取（通过 `lock_shared` 、 `try_lock_shared` ）。在一个线程内，同一时刻只能获取一个锁（共享或独占性）。共享互斥体在能由任何数量的线程同时读共享数据，但一个线程只能在无其他线程同时读写时写同一数据时特别有用。

- 排他性锁定
 - `lock()`: 锁定互斥，若互斥则阻塞
 - `try_lock()`: 尝试锁定互斥，若互斥不可用则返回
 - `unlock()`: 解锁互斥
- 共享锁定
 - `lock_shared()`:为共享所有权锁定互斥，若互斥不可用则阻塞
 - `try_lock_shared()`: 尝试为共享所有权锁定互斥，若互斥不可用则返回
 - `unlock_shard()`: 解锁互斥

2.3 各种锁介绍

前面已经简单介绍了 `std::lock_guard`，接下来将介绍其他类型的常用锁。

2.3.1 std::unique_lock -- 更加灵活的锁

`std::unique_lock`比 `std::lock_guard`灵活很多，效率上差一点，内存占用多一点，可移动，但不可复制。

`std::unique_lock`构造时除了接受第一个参数`mlock`, 可接受第二个参数，指定锁定策略：

- `defer_lock_t`: 不获得互斥的所有权，即仅仅构造`unique_lock`与`mlock`关联，但是并不上锁
- `try_to_lock_t`: 尝试获得互斥的所有权而不阻塞，可以使用成员函数 `bool owns_lock()`检测是否上锁成功
- `adopt_lock_t`: 假设调用方线程已拥有互斥的所有权，即构造构造`unique_lock`与`mlock`关联之前，已经对`mlock`加锁

show me the case

```
//defer_lock_t
void transfer(bank_account &from, bank_account &to, int amount)
{
    // 锁定两个互斥而不死锁
    std::lock(from.m, to.m); //对from.m和to.m加锁
    // 保证二个已锁定互斥在作用域结尾解锁
    std::lock_guard<std::mutex> lock1(from.m, std::adopt_lock);
    std::lock_guard<std::mutex> lock2(to.m, std::adopt_lock);

    from.balance -= amount;
    to.balance += amount;
}

//try_to_lock_t
for (int i = 1; i <= 5000; i++) {
    std::unique_lock<std::mutex> munique(mlock, std::try_to_lock);
    if (munique.owns_lock() == true) { //加锁成功
        s += i;
    }
    else {
        // 执行一些没有共享内存的代码
    }
}

//adopt_lock_t
void transfer(bank_account &from, bank_account &to, int amount)
{
    // 保证二个已锁定互斥在作用域结尾解锁
    std::unique_lock<std::mutex> lock1(from.m, std::defer_lock);
    std::unique_lock<std::mutex> lock2(to.m, std::defer_lock);
    std::lock(lock1, lock2); //对互斥量加锁

    from.balance -= amount;
    to.balance += amount;
}
```

不同域中互斥量所有权的传递

`std::unique_lock`实例没有与自身相关的互斥量，一个互斥量的所有权可以通过移动操作，在不同的实例中进行传递。某些情况下，这种转移是自动发生的，例如:当函数返回一个实例；另些情况下，需要显式的调用 `std::move()` 来执行移动操作。

```
std::unique_lock<std::mutex> get_lock()
{
    extern std::mutex some_mutex;
    std::unique_lock<std::mutex> lk(some_mutex); //构造unique_lock
    prepare_data();
    return lk; //返回unique_lock的指针，离开作用域lk不会被销毁，而是move到域外
}
void process_data()
{
    std::unique_lock<std::mutex> lk(get_lock()); // 获得锁所有权
    do_something();
}
```

2.3.2 单次调用加锁

`std::once_flag`是 `std::call_once`的辅助类。传递给多个 `std::call_once` 调用的 `std::once_flag` 对象允许那些调用彼此协调，从而只令调用之一实际运行完成。

`std::call_once`准确执行一次可调用 (Callable) 对象 `f`，即使同时从多个线程调用。

- 若在调用 `call_once` 的时刻，`flag` 指示已经调用了 `f`，则 `call_once` 立即返回（称这种对 `call_once` 的调用为消极）
- 否则调用可调用 (Callable) 对象`f`执行
 - 若该调用抛异常，则传播异常给 `call_once` 的调用方，并且不翻转 `flag`，以令其他调用将得到尝试（称这种对 `call_once` 的调用为异常）。
 - 若该调用正常返回（称这种对 `call_once` 的调用为返回），则翻转 `flag`，并保证以同一 `flag` 对 `call_once` 的其他调用为消极。

```
std::once_flag flag1, flag2;

void simple_do_once()
{
    std::call_once(flag1, [](){ std::cout << "Simple example: called once\n"; });
}

void may_throw_function(bool do_throw)
{
    if (do_throw) {
```

```

    std::cout << "throw: call_once will retry\n"; // 这会出现多于一次，因为出现异常的
    // 话，其他的调用会得到尝试
    throw std::exception();
}
std::cout << "Didn't throw, call_once will not attempt again\n"; // 如果未发生异
// 常，则保证函数只会被调用一次
}

void do_once(bool do_throw)
{
    try {
        std::call_once(flag2, may_throw_function, do_throw);
    }
    catch (...) {
    }
}

int main()
{
    std::thread st1(simple_do_once); // 1
    std::thread st2(simple_do_once); // 2
    std::thread st3(simple_do_once); // 3
    std::thread st4(simple_do_once); // 4
    st1.join();
    st2.join();
    st3.join();
    st4.join();

    std::thread t1(do_once, true); //5
    std::thread t2(do_once, true); //6
    std::thread t3(do_once, false); //7
    std::thread t4(do_once, true); //8
    t1.join();
    t2.join();
    t3.join();
    t4.join();
}

```

可能的结果:

Simple example: called once //1、2、3、4只会成功调用一次

throw: call_once will retry //5、6、8会触发异常，现在的结果可能是5、6触发两次异常，7调用成功，8不会再触发调用而是立刻返回

throw: call_once will retry

Didn't throw, call_once will not attempt again

2.3.3 同时加锁多个互斥量

2.3.3.1 std::lock

锁定给定的可锁定 (Lockable) 对象 lock1 、 lock2 、 ... 、 lockn ，用免死锁算法避免死锁。以对 lock 、 try_lock 和 unlock 的未指定系列调用锁定对象。若调用 lock 或 unlock 导致异常，则在重抛前对任何已锁的对象调用 unlock。

`std::lock`锁住的锁不会自动释放锁，需要手动解锁，因此 `std::lock`常与 `std::lock_guard`或者 `std::unique_lock`结合使用，比如

```
std::lock(m1, m2); //此处加锁，构造的lock_guard无需上锁，离开作用域自动解锁
std::lock_guard lock1(m1, std::adopt_lock);
std::lock_guard lock2(m2, std::adopt_lock);

或者
std::unique_lock lock1(m1, std::defer_lock);
std::unique_lock lock2(m2, std::defer_lock);
std::lock(lock1, lock2) //构造的unique_lock未上锁，此处加锁，离开作用域自动解锁
```

`std::try_lock`的作用是与 `std::lock`相似，可以同时多个互斥量加锁而不会死锁，通过以从头开始的顺序调用 `try_lock`。

若调用 `try_lock` 失败，则不再进一步调用 `try_lock`，并对任何已锁对象调用 `unlock`，返回锁定失败对象的 0 底下标。成功时为 -1，否则为锁定失败对象的 0 底下标值。

若调用 `try_lock` 抛出异常，则在重抛前对任何已锁对象调用 `unlock`。

2.3.3.2 std::scoped_lock(C++ 17)

类 `scoped_lock` 是提供便利 RAII 风格机制的互斥包装器，它在作用域块的存在期间占有一或多个互斥。

创建 `scoped_lock` 对象时，它试图取得给定互斥的所有权。控制离开创建 `scoped_lock` 对象的作用域时，析构 `scoped_lock` 并释放互斥。若给出数个互斥，则使用免死锁算法，如同以 `std::lock`。*show me the case*

```
std::scoped_lock lock(m1, m2);

//等价代码1
std::lock(m1, m2);
std::lock_guard<std::mutex> lk1(m1, std::adopt_lock);
std::lock_guard<std::mutex> lk2(m2, std::adopt_lock);

//等价代码2
std::unique_lock<std::mutex> lk1(m1, std::defer_lock);
std::unique_lock<std::mutex> lk2(m2, std::defer_lock);
std::lock(lk1, lk2);
```

2.3.4 共享锁(C++ 14)

`std::shared_lock`会以共享模式锁定关联的共享互斥（`std::unique_lock`可用于以排他性模式锁定）。

```
class SaferCounter {
    std::shared_mutex mutex;
    unsigned int get() const {
        std::shared_lock<std::shared_mutex> lock(mutex); //获取共享锁，内部执行
        mutex.lock_shared()
        return value_; //lock 析构，执行mutex.unlock_shared();
    }

    unsigned int increment() {
        std::unique_lock<std::shared_mutex> lock(mutex) //获取独占锁，内部执行
        mutex.lock()
        value++;
        return value; //lock 析构，执行mutex.unlock();
    }
}
```

3 同步并发操作

何时需要线程同步

- 线程完成前，需要等待另一个线程执行
- 线程需要等待特定事件发生
- 线程等待某个条件变为true

线程同步的方式

1. 持续检查共享标记

```
void wait_for_flag() {
    std::unique_lock lock(m);

    while (!flag) {
        lock.unlock();
        lock.lock();
    }

    do_something();
}
```

2. 等待线程在检查间隙

```

void wait_for_flag() {
    std::unique_lock lock(m);

    while (!flag) {
        lock.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(100)); //休眠
        lock.lock();
    }

    //going on next
}

```

3. 条件变量(condition variable)

3.1 条件变量使用方式

目前有以下两种方式，两者都需要与一个互斥量才能工作。

- `std::condition_variable`: 仅限于和 `std::mutex` 一起工作
- `std::condition_variable_any`: 可以和任何满足最低标准的互斥量一起工作，更加通用，但是体积、性能、系统资源会产生额外的开销

```

std::mutex lock;
std::queue<data_set> data_queue;
std::condition_variable data_cond;

void data_preparation_thread() {
    while (more_data_to_prep()) {
        data_set const data = prep_data();
        std::lock_guard l(lock);
        data_queue.push(data);
        /**
         1. notify_one()触发一个正在执行wait()的线程，去检查条件和wait()函数的返回状态
         2. 另一种可能是，很多线程等待同一事件，对于通知他们都需要做出回应，需要使用
        notify_all()
        **/
        data_cond.notify_one();
    }
}

void data_processing_thread() {
    while (true) {
        std::unique_lock l(lock); //后续需要unlock，因此不能用lock_guard
        /*1. 在条件满足的情况下，从wait()返回并继续持有锁。当条件不满足时，线程将对互斥量
        解锁，并且重新开始等待
         2. 当等待线程重新获取互斥量并检查条件时，如果它并非直接响应另一个线程的通知，这
        就是所谓的“伪唤醒”(spurious wakeup)。*/
        data_cond.wait(l, []{return !data_queue.empty();});
    }
}

```

另一种形式.

```
if (data.queue.empty()) {
    data_cond.wait(1);
}
**/
data_set data = data_queue.front();
data_queue.pop;
l.unlock();

process(data);

if (is_last_data(data)) {
    break;
}
}
}
```

3.2 使用期待处理一次性事件

当等待线程只等待一次，当条件为true时，它就不会再等待条件变量了，那么这种情况下使用条件变量会存在一定的浪费。

C++将这种一次性事件称为“期望”(future)。当一个线程需要等待一个特定的一次性事件时，future有以下集中应用方式：

1. 这个线程可以周期性(较短的周期)的等待或检查，事件是否触发(检查信息板)；
2. 在检查期间也可以执行其他任务；
3. 在等待任务期间它可以先执行另外一些任务，直到对应的任务触发，而后等待期望的状态会变为“就绪”(ready)

C++有两种期望类型：`std::future`和`std::shared_future`。`std::future`的实例只能与一个指定事件相关联，而`std::shared_future`的实例就能关联多个事件。后者的实现中，所有实例会在同时变为就绪状态，并且他们可以访问与事件相关的任何数据。

3.2.1 std::future

比如需要一个长时间的运算，但是现在并不需要关注这个值，在需要时再去获取，我们来看条件变量的方式

```
bool result_ok();
int get_result(); //假设result非0
int result = 0;
std::mutex lock;
```

```

void main() {
    std::unique_lock l(lock); //1. 互斥量
    while(!result) {
        wait(1, result_ok); //2. 条件变量, 阻塞等待
        result = get_result;
    }
}

//3. 独立计算result的线程
std::unique_lock l(lock);
calculate_result();
notify_all();

```

接下来是future的方式:

```

/*当任务的结果你不着急要时, 你可以使用std::async启动一个异步任务。与std::thread对象等待运行方式的不同, std::async会返回一个std::future对象, 这个对象持有最终计算出来的结果。当你需要这个值时, 你只需要调用这个对象的get()成员函数; 并且直到“期望”状态为就绪的情况下, 线程才会阻塞; 之后, 返回计算结果*/
void main() {
    std::future<int> future_result = std::async(calculate_result);
    do_something() //如果暂时不需要结果, 可以做些其他事情
    result = future_result.get(); //需要的时候就去获取结果, 如果future未ready则阻塞
}

```

更多请查看[std::future](#)

async简述 函数模板 **async** 异步地运行函数 **f**（潜在地在可能是线程池一部分的分离线程中），并返回最终将保有该函数调用结果的 **std::future**。

async的构造方式:

1. 传入函数+参数: `auto f2=std::async(bar,"goodbye")`
2. 传入成员函数指针+成员类的对象+成员函数参数: `auto f1=std::async(&X::foo,&x,42,"hello")`
3. 传入引用: `std::async(baz,std::ref(x));` // 调用**baz(x)** 更多构造方式见[std::async](#)

在函数调用之前, 向**std::async**传递一个额外参数, 参数的类型是**std::launch**, 有以下几种取值:

- 若设置 **async** 标志, 即 `std::launch::async`, 则 **async** 在新的执行线程（初始化所有线程局域对象后）执行可调用对象 **f**, 如同产出 `std::thread(std::forward<F>(f), std::forward<Args>(args)...)` , 除了若 **f** 返回值或抛出异常, 则于可通过 **async** 返回给调用方的 **std::future** 访问的共享状态存储结果。

- 若设置 `deferred` 标志，即 `std::launch::deferred`，则 `async` 以同 `std::thread` 构造函数的方式转换 `f` 与 `args...`，但不产出新的执行线程。而是进行惰性求值：在 `async` 所返回的 `std::future` 上首次调用非定时等待函数，将导致在当前线程（不必是最初调用 `std::async` 的线程）中，以 `args...`（作为右值传递）的副本调用 `f`（亦作为右值）的副本。将结果或异常置于关联到该 `future` 的共享状态，然后才令它就绪。对同一 `std::future` 的所有后续访问都会立即返回结果。
- 若 `policy` 中设置了 `std::launch::async` 和 `std::launch::deferred` 两个标志，则进行异步执行还是惰性求值取决于实现

```
auto f6=std::async(std::launch::async,Y(),1.2); // 在新线程上执行
auto f7=std::async(std::launch::deferred,baz,std::ref(x)); // 在wait()或get()调用时
执行
```

3.2.2 std::shared_future

`std::future` 所引用的共享状态不与另一异步返回对象共享，`std::future`模型独享同步结果的所有权，并且通过调用 `get()`函数，一次性的获取数据，这就让并发访问变的毫无意义——只有一个线程可以获取结果值，因为在第一次调用 `get()`后，就没有值可以再获取了，再次调用 `get()`会抛出异常而 `std::shared_future`允许多个线程等候同一共享状态，可用于同时向多个线程发信，类似

`std::condition_variable::notify_all()`，多个对象可以引用同一关联“期望”的结果，简而言之，`std::shared_future`中共享状态可以被 `get()`多次。注意在每一个 `std::shared_future`的独立对象上成员函数调用返回的结果还是不同步的，所以为了在多个线程访问一个独立对象时，避免数据竞争，必须使用锁来对访问进行保护。

```
int queryNumber();
void doSomething(char c, shared_future<int> f);
void check() {
    try {
        shared_future<int> f = std::async(queryNumber);

        auto f1 = std::async(std::launch::async, doSomething, '.', f);
        auto f2 = std::async(std::launch::async, doSomething, '+', f);

        f1.get();
        f2.get();
    }
    catch (const std::exception& e) {
        std::cout << "Exception: " << e.what << endl;
    }
}
```

3.3 更多的使用 `future` 的方式

除了前面 `std::async` 的使用方式，还可以使用 `std::package_task` 和 `std::promise`。`std::package_task` 可以封装一个可调用对象，待后续调用，而 `std::promise` 则可以封装一个值，待后续使用。

3.3.1 `std::package_task`

`std::packaged_task<>` 对一个函数或可调用对象，绑定一个期望。当 `std::packaged_task<>` 对象被调用，它就会调用相关函数或可调用对象，将期望状态置为就绪，返回值也会被存储为相关数据。它包装任何可调用 (Callable) 目标，包括函数、lambda 表达式、bind 表达式或其他函数对象，使得能异步调用它。

`std::packaged_task<>` 的模板参数是一个函数签名，如：

```
int f(int x, int y) { return std::pow(x,y); }
std::packaged_task<int(int,int)> task(f)
```

使用 `std::packaged_task` 关联的 `std::future` 对象保存的数据类型是可调用对象的返回结果类型，如示例函数的返回结果类型是 `int`，那么声明为 `std::future<int>`，而不是 `std::future<int(int)>`。

```
int Add(int x, int y);

void task_lambda() {
    int ret;
    std::packaged_task<int(int, int)> task([](int a, int b){return a + b;}); //使用
    lambda表达式包装可调用函数

    task(2, 10); //启动任务，非异步

    std::future<int> result = task.get_future();
    ret = result.get(); //获取共享状态的值

    task.reset(); //重置共享状态
    result = task.get_future();

    thread td(std::move(task), 2, 10) //异步启动
    ret = result.get();
}
```

3.3.2 std::promise

类模板 `std::promise` 提供存储值或异常的设施，之后通过 `std::promise` 对象所创建的 `std::future` 对象异步获得结果。注意 `std::promise` 只应当使用一次。

`promise` 是 `promise-future` 交流通道的“推”端：存储值于共享状态的操作同步于任何在共享状态上等待的函数（如 `std::future::get`）的成功返回。其他情况下对共享状态的共时访问可能冲突：例如，`std::shared_future::get` 的多个调用方必须全都是只读，或提供外部同步。

一对 `std::promise/std::future` 在期望上可以阻塞等待线程，同时，提供数据的线程可以使用组合中的“承诺”来对相关值进行设置，以及将“期望”的状态置为“就绪”。

可以通过 `get_future()` 成员函数来获取与一个给定的 `std::promise` 相关的 `std::future` 对象，就像是与 `std::packaged_task` 相关。当“承诺”的值已经设置完毕（使用 `set_value()` 成员函数），对应“期望”的状态变为“就绪”，并且可用于检索已存储的值。当你在设置值之前销毁 `std::promise`，将会存储一个异常。

在调用 `std::future::get()` 时，如果 `std::future` 对象状态不是 `ready`，则调用的地方将一直阻塞等待。

show me the case

```
int thread_task(std::promise<int> & pro, int i) {
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    pro.set_value(i); //提醒 future
    return 0;
}

void main() {
    std::promise<int> pro; //promise<int> 在线程间传递结果
    std::thread mythread(thread_task, std::ref(pro), 5);
    mythread.join();

    std::future<int> result = pro.get_future();
    //future::get() 将等待直至该 future 拥有合法结果并取得它
    std::cout << result.get() << std::endl;
}
```

4 原子操作

4.1 内存顺序

内存顺序描述了计算机 CPU 获取内存的顺序，内存的排序既可能发生在编译器编译期间，也可能发生在 CPU 指令执行期间。

为了尽可能地提高计算机资源利用率和性能，编译器会对代码进行重新排序，CPU 会对指令进行重新排序、延缓执行、各种缓存等等，以达到更好的执行效果。当然，任何排序都不能违背代码本身所表达的意义，并且在单线程情况下，通常不会有任何问题。

但是在多线程环境下，比如无锁（lock-free）数据结构的设计中，指令的乱序执行会造成无法预测的行为。

内存栅栏： 内存栅栏是一个令 CPU 或编译器在内存操作上限制内存操作顺序的指令，通常意味着在 **barrier** 之前的指令一定在 **barrier** 之后的指令之前执行。

4.2 概念解释

线程间同步和内存顺序决定表达式的求值和副效应如何在不同的执行线程间排序

先序于在同一个线程中，在同一线程中，求值 A 可以先序于求值 B

求值顺序： 求值任何表达式的任何部分，包括求值函数参数的顺序都是未说明的（除了下列的一些例外）。编译器能以任何顺序求值任何操作数和其他子表达式，并且可以在再次求值同一表达式时选择另一顺序。C++ 中无从左到右或从右到左求值的概念。

顺序“按顺序早于 (sequenced-before)”是同一线程中的求值之间的非对称的、传递的对偶关系。

- 若 A 按顺序早于 B，则 A 的求值将在 B 的求值开始前完成。
- 若 A 不按顺序早于 B 而 B 按顺序早于 A，则 B 的求值将在 A 的求值开始前完成。
- 若 A 不按顺序早于 B 而 B 不按顺序早于 A，则存在两种可能：
 - A 与 B 的求值是无顺序 (unsequenced) 的：它们能以任何顺序进行，并可能重叠（在同一执行线程内，编译器可以将组成 A 与 B 的 CPU 指令交错）
 - A 与 B 的求值是顺序不确定 (indeterminately sequenced) 的：它们可以任意顺序进行但不可重叠，A 在 B 前完成，或 B 在 A 前完成。下次求值相同表达式时顺序可以相反。

具体的规则见[按顺序早于规则](#)

消费操作 带 `memory_order_consume` 或更强标签的原子加载是消费操作。

获得操作 带 `memory_order_acquire` 或更强标签的原子加载是获得操作。互斥体 (Mutex) 上的 `lock()` 操作亦为获得操作

释放操作 带 `memory_order_release` 或更强标签的原子存储是释放操作。互斥体 (Mutex) 上的 `unlock()` 操作亦为释放操作。

4.3 memory order

在 C11/C++11 中，引入了六种不同的 `memory order`，可以让程序员在并发编程中根据自己需求尽可能降低同步的粒度，以获得更好的程序性能: `relaxed`, `consume`, `acquire`, `release`, `acq_rel`, `seq_cst`

- **`memory_order_relaxed`** 没有同步或顺序制约，只保证当前操作的原子性，不考虑线程间的同步，其他线程可能读到新值，也可能读到旧值。

```
std::atomic<int> x = 0;
std::atomic<int> y = 0;

//thread 1
r1 = y.load(memory_order_relaxed); //A
x.store(r1, memory_order_relaxed); //B

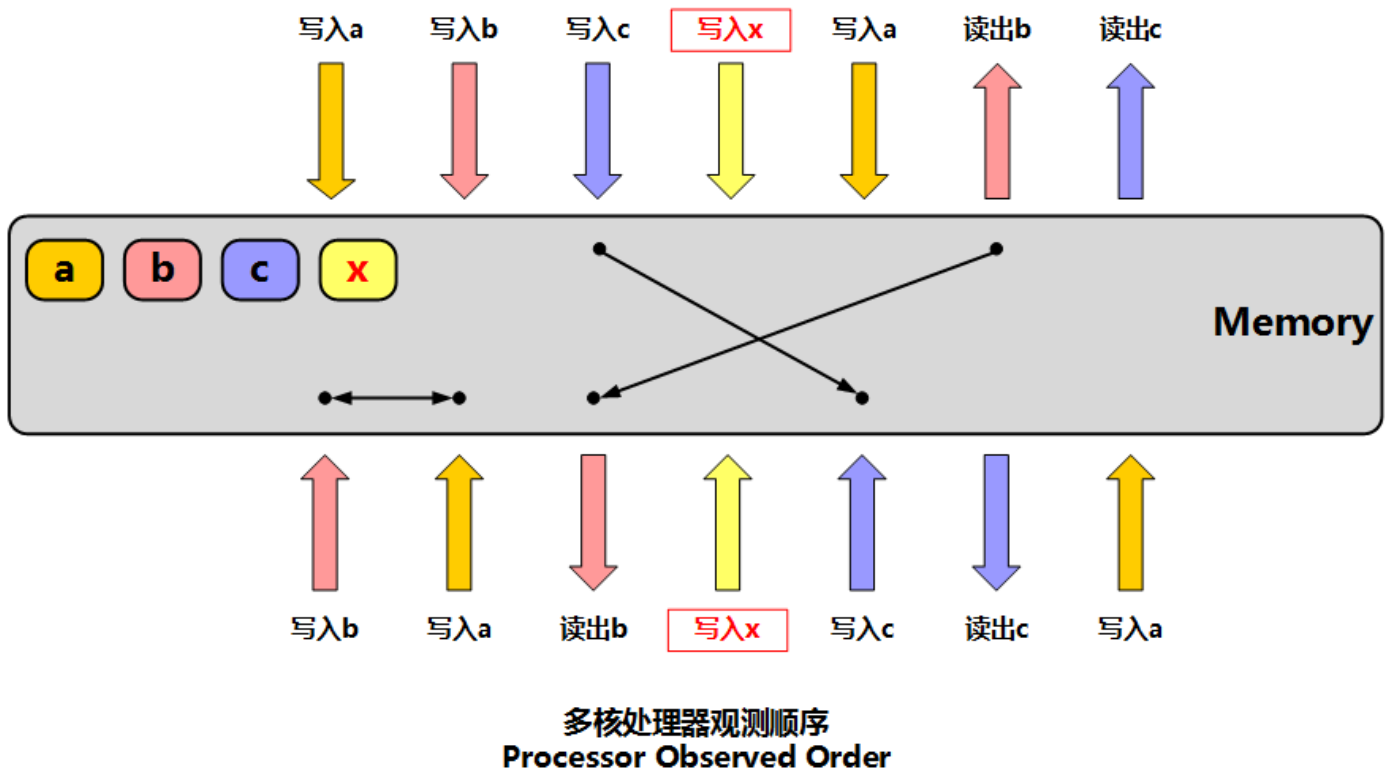
//thread 2
r2 = x.load(memory_order_relaxed); //C
y.store(42, memory_order_relaxed); //D
```

允许产生结果: `r1 == 42 && r2 == 42`,虽然thread 1中A先于B，thread 2中C先于D，按常理C不应该读到D设置于y中的值，但是实际情况却没有避免D先于A，也就是说D在y上的修改，可能可见于thread1中加载A，同时B在x的存储可能可见于thread 2中的加载

C。

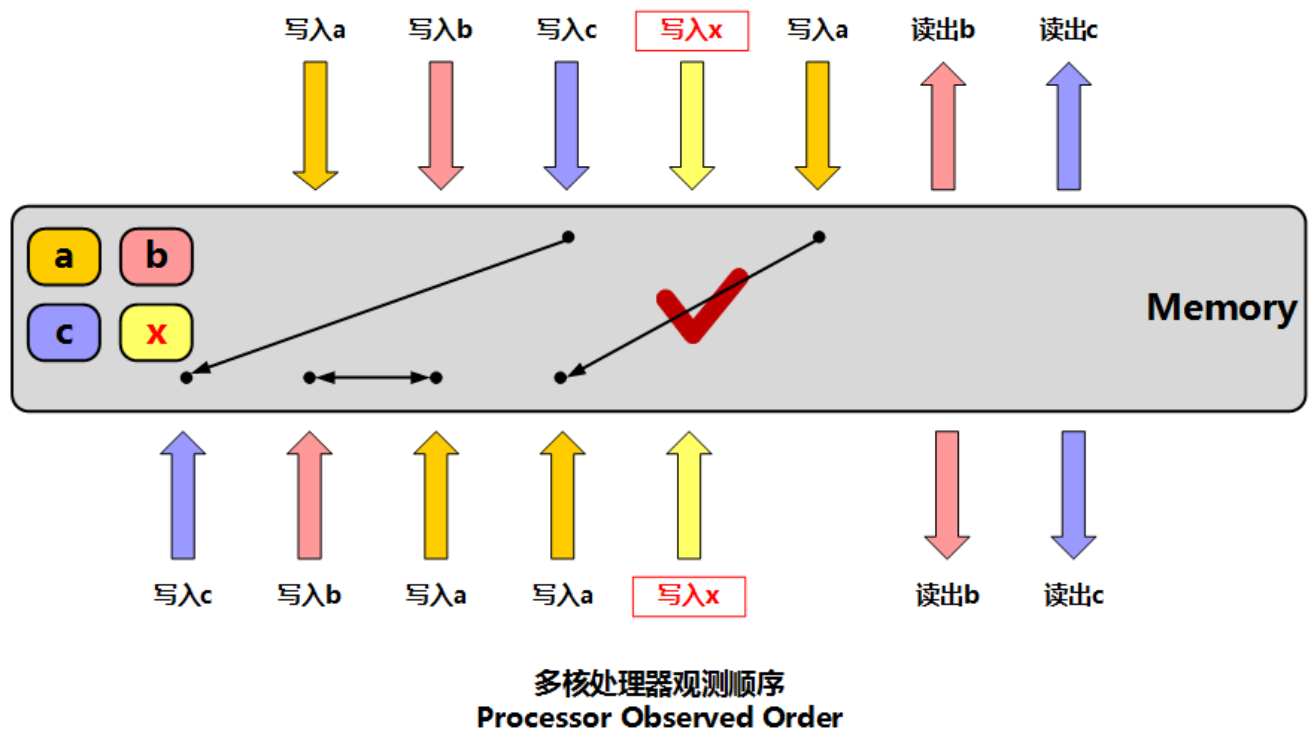
`std::memory_order_relaxed`

指令序
(Program Order)



如上图所示，假设：a, b, c 分别为普通变量，而 x 为 `atomic` 类型的变量，当在写入 x 时，设置 `memory_order_relaxed` 时，写入 a，写入 b 的顺序，在另外的线程上看到的，完全可能是相互调换的，写入 c 的位置，完全也有可能由出现在写入 x 之前，而在另外一个线程上看到的，确实在写入 x 之后，同时，读出 b 的动作，完全有可能从写入 x 之后，编程在另外一个线程上看，是在写入 x 之前。也就是，各种乱序，都是被允许的。

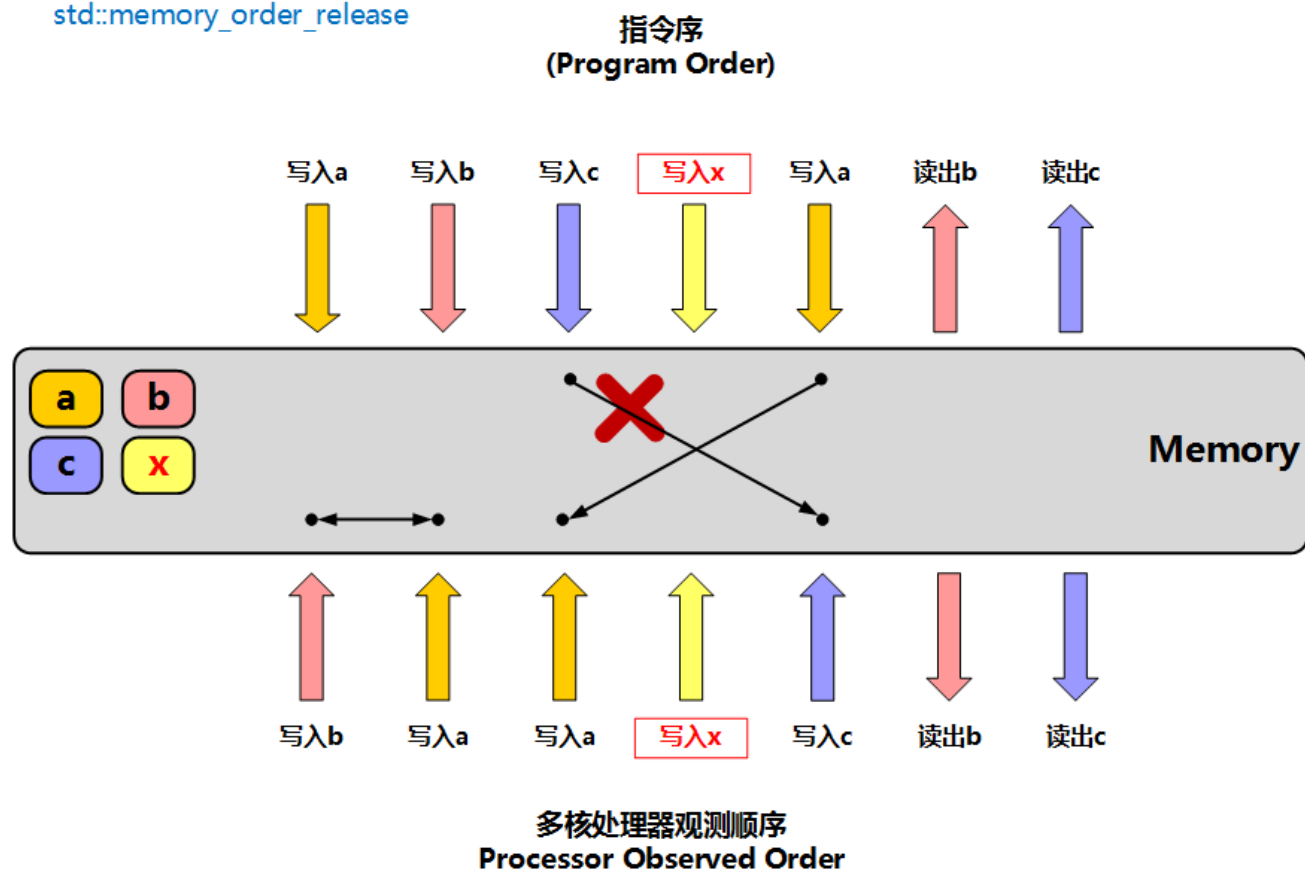
- **`memory_order_consume`**有此内存顺序的加载操作，在其影响的内存位置进行消费操作：当前线程中依赖于当前加载的该值的读或写不能被重排到此加载前。其他释放同一原子变量的线程的对数据依赖变量的写入，为当前线程所可见。在大多数平台上，这只影响到编译器优化
- **`memory_order_release`**单向的“释放”内存屏障，释放操作，当前线程中的读或写不能被重排到此存储后。当前线程的所有写入，可见于加载该同一原子变量的其他线程。

指令序
(Program Order)

如上图，原子操作写入的情况下，原子操作之后的读写操作，在另外的线程（加载了该原子变量的线程）看来，可以重排到原子操作之前（通俗点说，就是另外线程可能认为当前线程是先执行的写入的 **a** 操作，再执行的写入 **x** 操作）。但是，如下图所示，原子操作之前的读、写操作，计算机系统必须保证，在另外一个线程看来，他是在原子操作写入之前就被写入了，不能是在原子操作之后才写入（通俗点说，也就是另外一个线程，不能认为当前线程先写入的 **x**，再写入的 **c**，他一定要

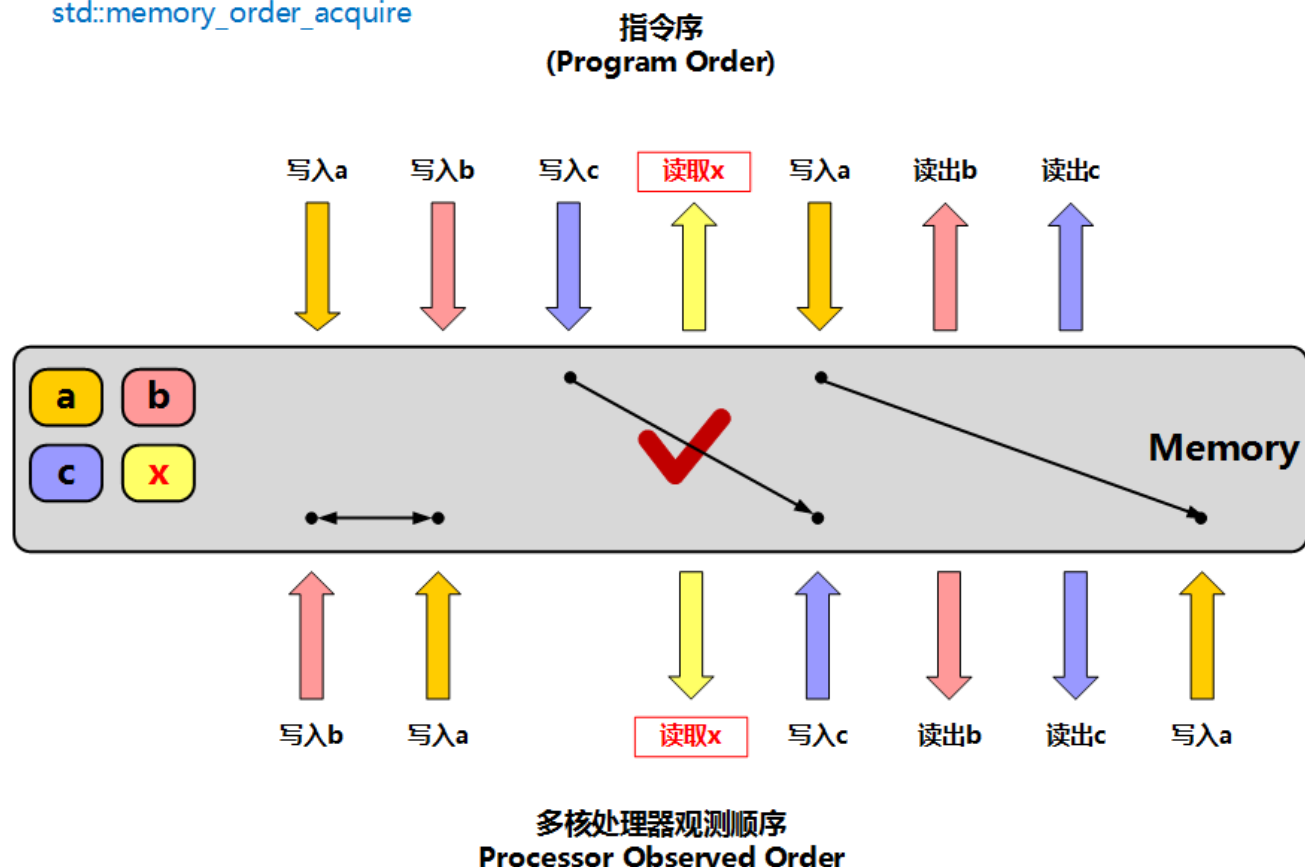
看到的是先写入 c，再写入的 x)：

`std::memory_order_release`



- **memory_order_acquire** 单向的”加载”内存屏障, 加载操作, 当前线程中读或写不能被重排到此加载前。其他释放同一原子变量的线程的所有写入, 为当前线程所可见。

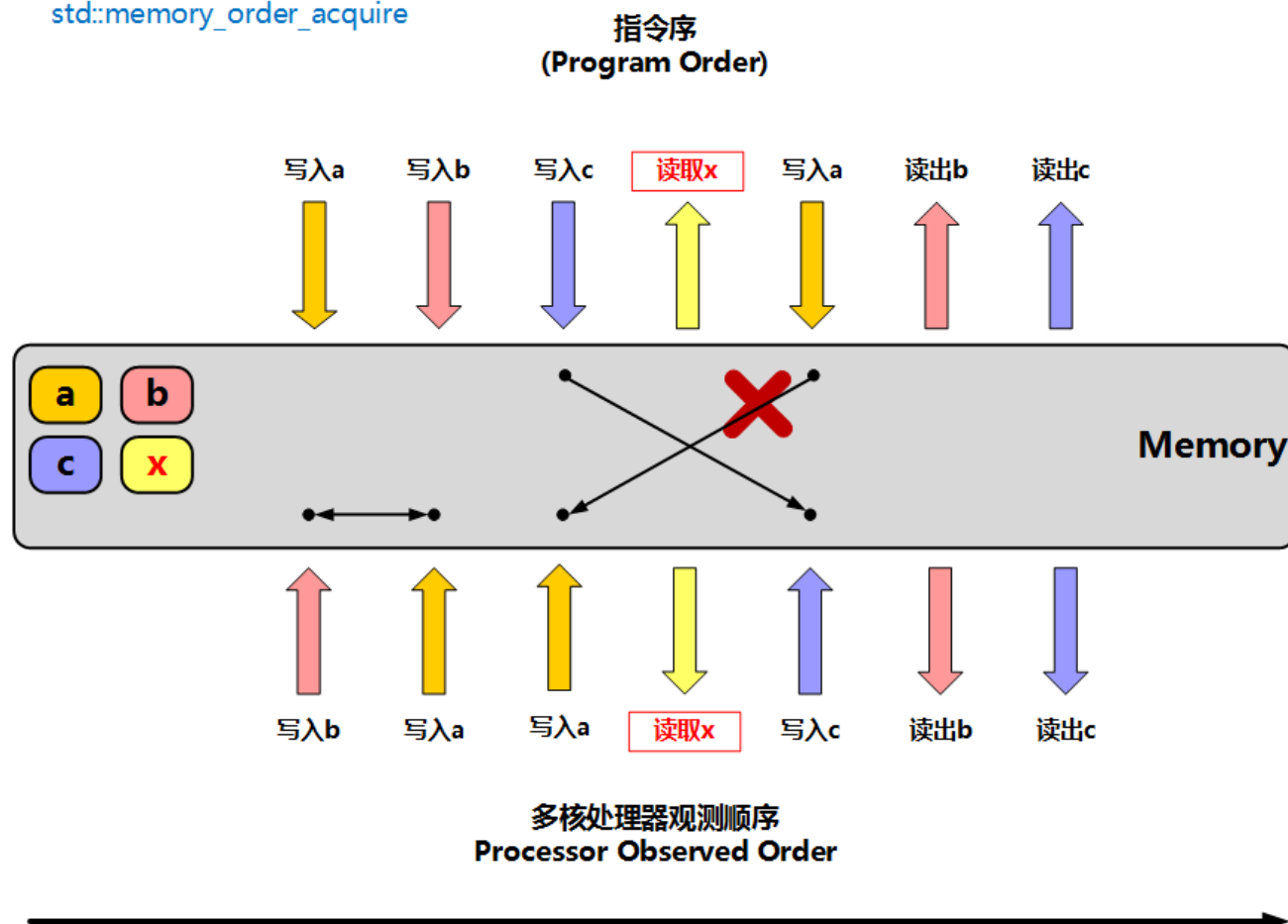
`std::memory_order_acquire`



如上图所示，对 x 的读操作前的读、写操作，在另外的线程看来（或者说实际的运

行顺序），可以被重排到对 x 的读操作后，(通俗点说，就是其他线程可以认为是认为当前线程先读取的 x ，再写入的 c)。然而如下图所示，对 x 的读操作后的读、写操作，在另外的线程看来不允许被重排到 x 的读操作前（不允许其他线程看到当前线程先写入的 a ，再读取的 x ）。

`std::memory_order_acquire`

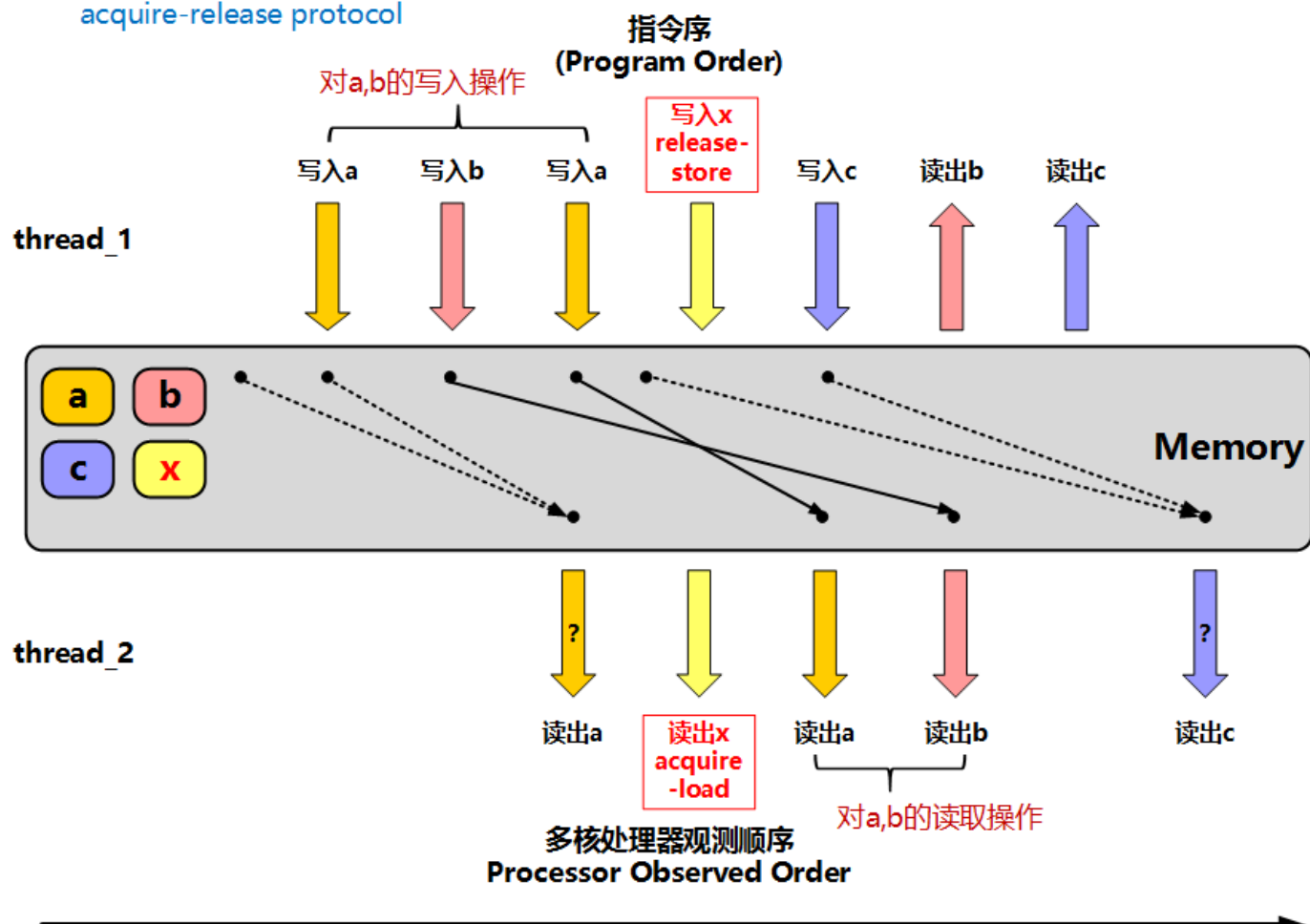


往往`memory_order_acquire`和`memory_order_release`是配合着一起使用的：

1. 线程 1 使用`memory_order_release`写入原子变量 x
2. 线程 2 使用`memory_order_acquire`读出原子变量 x

所有在线程 1 上，在写入 x 之前的写入操作，都将在线程 2 上，在读出 x 之后，被看到。使用单向“加载”+ 单向“释放”协议的场景往往是：

1. 线程 1，写入一些实际数据，接着通过将原子变量 x 设置为某个值 A （通过使用`memory_order_release`写入原子变量 x ）来“发布”这些数据。
2. 线程 2，通过读取并判断 x 已被设置为 A （通过使用`memory_order_acquire`来读取原子变量 x ），进而读取线程 1 实际“发布”的那些数据



如上图所示，thread_1 在 release 写入 x(值:A) 之前，写入了待发布的 a,b 的数据，而 thread_2，将在 acquire 读出 x 且为 A 之后，将读到 thread_1 发布的 a,b 的数据。同时，我们可以注意到，在 thread_2 上，在 acquire 读出 x 之前，如果对 a 进行读操作，我们是无法确认读到的 a 一定是 thread_1 在之前最后写入的 a，这里的顺序是不会被保证的，重排是被允许的。同时，在之后，读取 c，读到的是否为 thread_1 最后写入的 c，也是不确定的，因为，在 x 写入之后，thread_1 上又出现了一次写入，而如果在此之前，还有一次写入，这两次写入之间，是不存在限制，可能会被重排的。

thread_1 上有了 release_store，对于 a, b 的写入就一定会在 x 的改变之前，在 thread_2 上，就不会出现类似读出 c，的不确定性。thread_2 上有了 acquire_load，右侧的读出 a，就不会被重排读到左侧，而左侧读出 a 的不确定性，也不存在。thread_1 卡住的是：对于数据 a, b 的写入不能排到 x 的写入之后，thread_2 卡住的，是对于数据 a, b 的读取，不能排到读取 x 之前，这样，就保证了数据 a, b，与“信号量”x，之间，在 thread_1, thread_2 上的同步关系。

- **memory_order_acq_rel** 双向的“加载 - 释放”内存屏障，加载 - 释放操作，带此内存顺序的读 - 修改 - 写操作既是获得加载又是释放操作。没有操作能够从此操作之后被重排到此操作之前，也没有操作能够从此操作之前被重排到此操作之后，当然，具有这一限制的前提是，观察读写顺序的不同线程是使用的同一个原子变量并基于这一内存顺序。
- **memory_order_seq_cst** 最严的“顺序一致”内存屏障，此内存顺序的操作既是加载操作又是释放操作，没有操作能够从此操作之后被重排到此操作之前，也没有操

作能够从此操作之前被重排到此操作之后。带标签`memory_order_seq_cst`的原子操作不仅以与释放/加载顺序相同的方式排序内存（在一个线程中先发生于存储的任何结果都变成做加载的线程中的可见），还对所有拥有此标签的内存操作建立一个单独全序。`memory_order_seq_cst`比`memory_order_acq_rel`更强，`memory_order_acq_rel`的顺序保障，是要基于同一个原子变量的，也就是说，在这个原子变量之前的读写，不能重排到这个原子变量之后，同时这个原子变量之后的读写，也不能重排到这个原子变量之前。但是，如果两个线程基于`memory_order_acq_rel`使用了两个不同的原子变量 `x1`, `x2`，那在 `x1` 之前的读写，重排到 `x2` 之后，是完全可能的，在 `x1` 之后的读写，重排到 `x2` 之前，也是被允许的。然而，如果两个原子变量 `x1`, `x2`，是基于`memory_order_seq_cst`在操作，那么即使是 `x1` 之前的读写，也不能被重排到 `x2` 之后，`x1` 之后的读写，也不能重排到 `x2` 之前，也就说，如果都用`memory_order_seq_cst`，那么程序代码顺序 (Program Order) 就将会是你在多个线程上都实际观察到的顺序 (Observed Order)

5 参考

<https://zhuanlan.zhihu.com/p/45566448>

<https://www.dazhuanlan.com/yhao/topics/1192332>

https://zh.cppreference.com/w/cpp/atomic/memory_order#.E5.AE.BD.E6.9D.BE.E9.A1.BA.E5.BA.8F